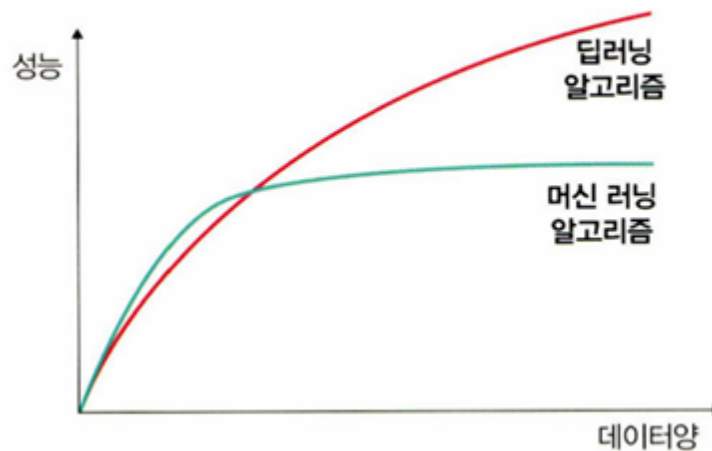


Week16_예습과제_장서연

8.1 성능 최적화

8.1.1 데이터를 사용한 성능 최적화

▼ 그림 8-1 데이터와 딥러닝, 머신 러닝 알고리즘의 성능 비교



- 최대한 많은 데이터 수집하기
- 데이터 생성하기
- 데이터 범위 조정하기
 - 활성화 함수 - 시그모이드 함수(0,1), 하이퍼볼릭 탄젠트 함수(-1,1)
- 정규화, 규제화, 표준화

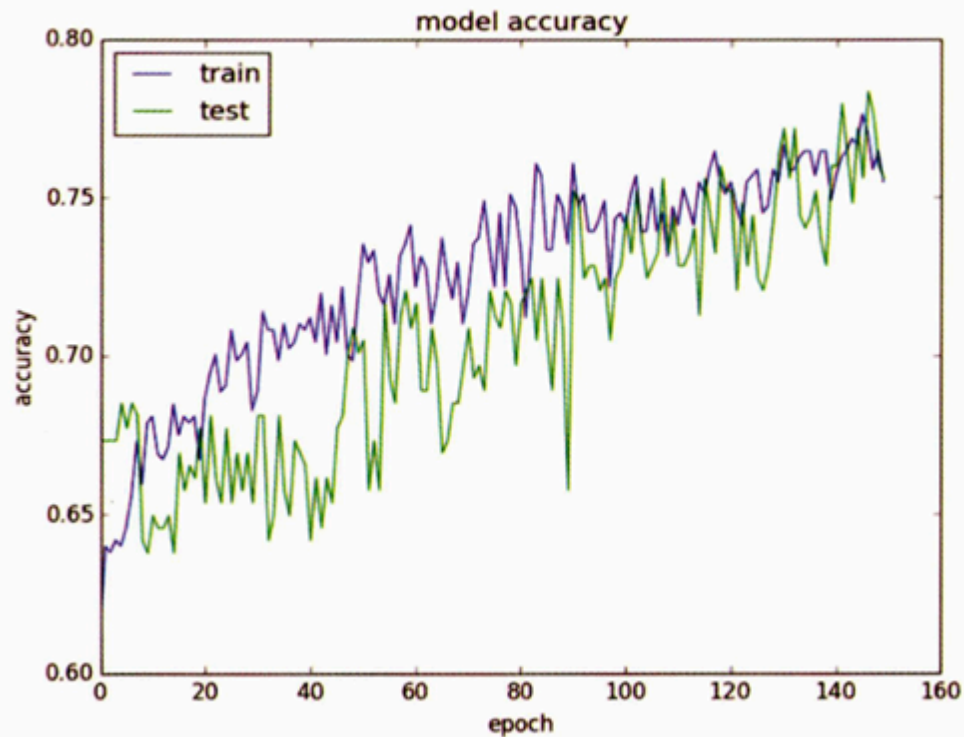
8.1.2 알고리즘을 이용한 성능 최적화

- 유사한 용도의 알고리즘들 비교
 - 예) 머신러닝 - SVM, k-NN
 - 예) 시계열 데이터 - RNN, LSTM, GRU

8.1.3 알고리즘 튜닝을 위한 성능 최적화

1. 진단

▼ 그림 8-2 알고리즘 성능 진단



- 성능 향상이 미묘해지는 원인 분석

- 예) 과적합

- 훈련 성능 >>> 검증 성능 → 규제화
 - 훈련 성능 ▼, 검증 성능 ▼ → 과소 적합
 - 훈련 성능 > 검증 성능 변곡점 존재 : 조기종료

2. 가중치

- 초기값 설정을 작은 난수로 설정
→ 오토 인코더 같은 비지도 학습을 이용해 사전 훈련 진행 → 지도 학습

3. 학습률

- 초기에 큰/작은 임의의 난수를 선택한 후 결과에 따라 조금씩 변경
- 계층 多 : 학습률 크게 설정
- 계층 少 : 학습률 작게 설정

4. 활성화 함수

- 변경하면 손실함수도 변경해야하므로 신중히 결정

- 예) 활성화 함수 : 시그모이드, 하이퍼볼릭 탄젠트 → 출력층 : 소프트맥스, 시그모이드

5. 배치와 에포크

- 큰 에포크, 작은 배치 설정이 트렌드
- 다양하게 테스트

6. 옵티마이저 및 손실 함수

- 확률적 경사하강법
- Adam, RMSProp 등 많이 사용
- 다양하게 테스트

7. 네트워크 구성 (네트워크 토폴로지, topology)

- 네트워크를 넓게 하기
- 네트워크를 좁고 깊게 하기
- 두가지를 결합
- 등등

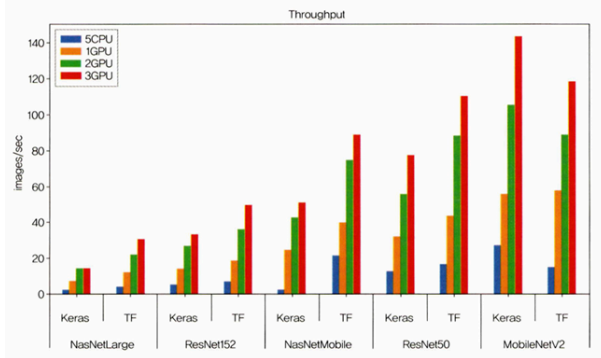
8.1.4 앙상블을 이용한 성능 최적화

- 모델 2개 이상 섞어서 사용
- 하이퍼파라미터에 대한 모든 경우의 수를 고려해야하므로 학습이 수백번까지 필요할 수 있음

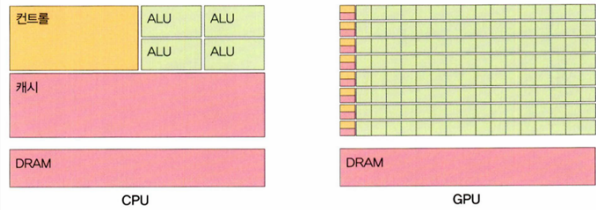
8.2 하드웨어를 이용한 성능 최적화

8.2.1 CPU, GPU 사용의 차이

▼ 그림 8-3 CPU와 GPU 성능 비교¹⁾



▼ 그림 8-4 CPU와 GPU의 구조 비교



✓ CPU

- ALU(연산) + control(명령 해석, 실행) + cash(데이터 저장)
- 직렬 처리 방식 : 명령어 입력 순서대로 데이터 처리
- 한번에 하나의 명령어만 처리
- ALU(Arithmetic Logic Unit) 개수가 많지 않아도 됨

✓ GPU

- 캐시 메모리 ↓, ALU 개수 ↑
- 병렬 처리 방식 : 서로 다른 명령어를 동시에 병렬적으로 처리
- 하나의 코어에 ALU 수백~수천개 장착

✓ CPU vs. GPU

- python, MATLAB : 직렬 연산 (행렬, 재귀)
→ **CPU**가 유리
- 역전파 : 복잡한 미적분
→ **GPU**가 유리
- 딥러닝에서는 GPU가 필

8.2.2 GPU를 이용한 성능 최적화

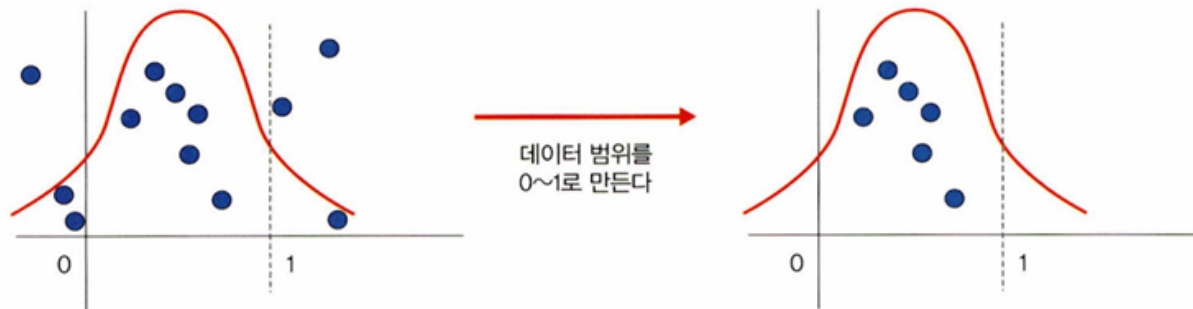
- CUDA(쿠다), cuDNN

8.3 하이퍼파라미터를 이용한 성능 최적화

8.3.1 배치 정규화를 이용한 성능 최적화

1 정규화(normalization)

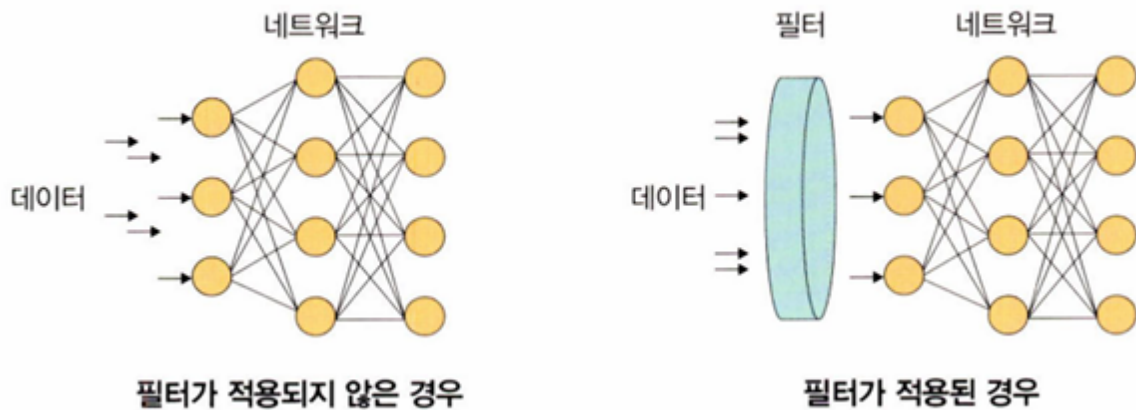
▼ 그림 8-31 정규화



- `MinMaxScaler()` 사용
- 수식 : $\frac{x - x_{min}}{x_{max} - x_{min}}$

2 규제화(regularization)

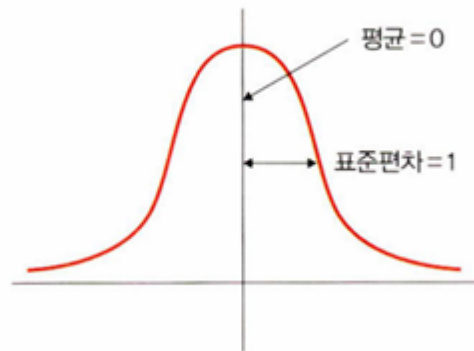
▼ 그림 8-32 규제화



- 모델 복잡도를 줄이기 위해 제약을 둬
- 종류
 - 드롭아웃
 - 조기 종료

3 표준화(standardization)

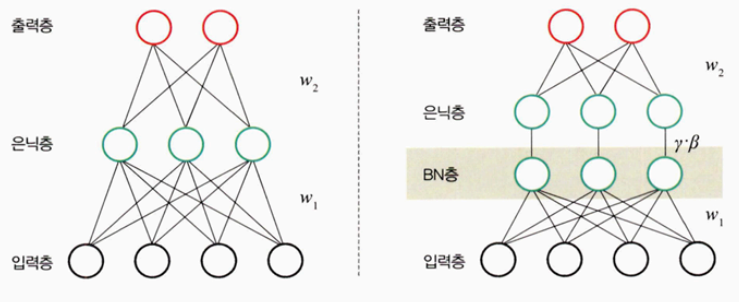
▼ 그림 8-33 표준화



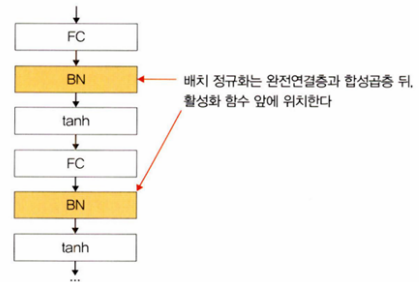
- 평균 0, 표준편차 1인 형태의 데이터로 만들기
- standard scaler, z-score 정규화
- 수식 : $\frac{x-\mu}{\sigma}$

4 배치 정규화(batch normalization)

▼ 그림 8-35 배치 정규화



▼ 그림 8-40 배치 정규화 위치



- 데이터 분포가 안정돼 속도를 높일 수 있음
- 완전 연결층과 합성곱층 뒤, 활성화 함수 앞에 위치
- 기울기 소멸, 기울기 폭발 문제 해결
 - 기울기 소멸 : 오차 정보를 역전파시키는 과정에서 기울기가 0에 가까워짐 → 학습 안됨
 - 기울기 폭발 : 학습 과정에서 기울기가 급격히 커짐
 - ReLU, 초깃값 튜닝, 학습률 조정 등으로 해결
- 기울기 소멸, 폭발의 원인 → 내부 공변량 변화(internal covariance shift) 로 정의
 - 네트워크의 각 층마다 활성화 함수 적용, 입력값들의 분포가 계속 변함

$$\mu\beta \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad \text{----①}$$

$$\sigma^2\beta \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu\beta)^2 \quad \text{----②}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu\beta}{\sqrt{\sigma^2\beta + \epsilon}} \quad \text{----③}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \Leftrightarrow BN_{\gamma, \beta}(x_i) \quad \text{----④}$$

$$\left(\begin{array}{l} \text{입력: } \beta = \{x_1, x_2, \dots, x_n\} \\ \text{학습해야 할 하이퍼파라미터: } \gamma, \beta \\ \text{출력: } y_i = BN_{\gamma, \beta}(x_i) \end{array} \right)$$

- 미니배치에 표준화와 유사한 방식 적용, 정규 분포화

- 매단계마다 활성화 함수를 거치며 데이터 분포가 일정해지므로 속도 향상

1. 미니 배치 평균 구하기
2. 미니 배치 분산, 표준 편차 구하기
3. 정규화
4. 스케일 조정

• 단점

1. 배치 크기가 작으면 정규화 값이 기존값과 다른 방향으로 학습될 수 있음

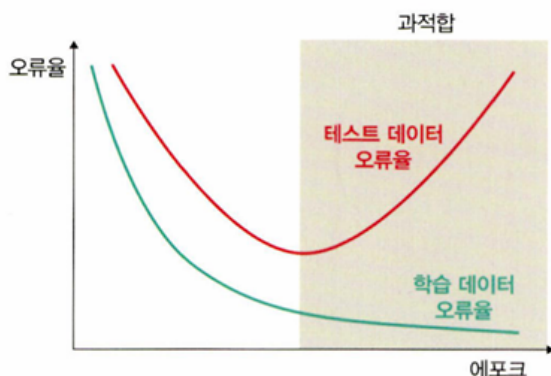
a. 예) 분산 = 0이면 정규화가 안됨

2. RNN = 네트워크 계층별로 미니 정규화 적용해야 함, 모델 복잡도 

→ 가중치 수정, 네트워크 구성 변경 등 수행

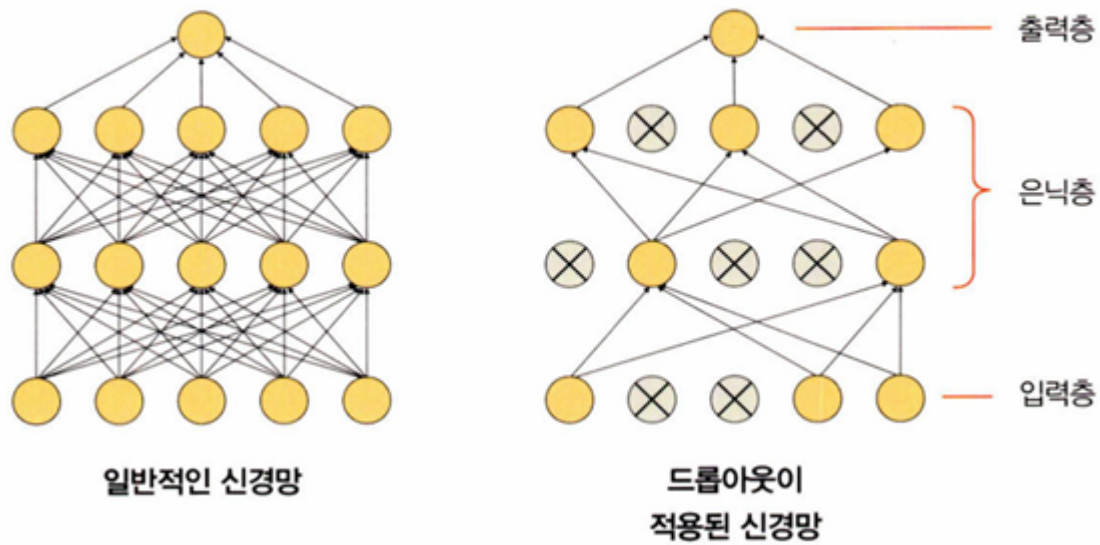
8.3.2 드롭아웃을 이용한 성능 최적화

▼ 그림 8-36 훈련과 오류율



- 과적합 : train dataset에 대해 학습이 지속되면 train accuracy는 상승하지만 test accuracy는 감소

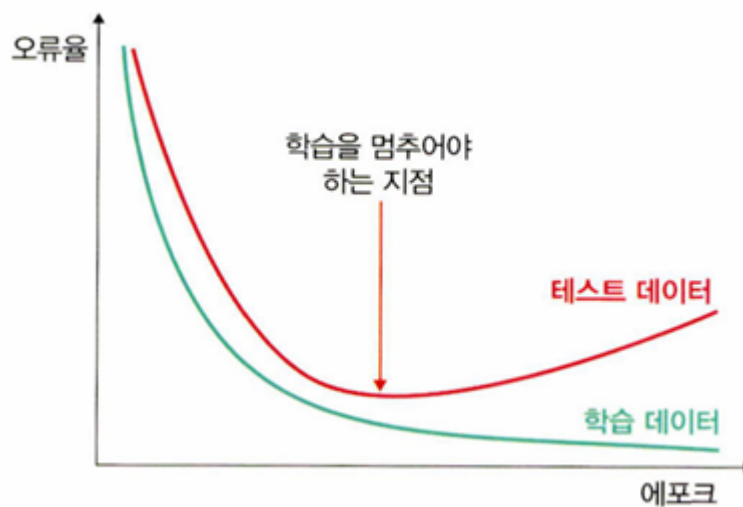
▼ 그림 8-37 드롭아웃



- 일정 비율의 뉴런만 사용하고 나머지 뉴런에 해당하는 가중치는 업데이트 ✖
- 단계마다 꺼지는 뉴런이 무작위로 달라짐
- 단점 : 학습 시간 증가
- 테스트 데이터로 평가 시 노드를 모두 사용해 출력, 드롭아웃 비율을 곱해 성능 평가

8.3.3 조기 종료를 이용한 성능 최적화

▼ 그림 8-50 조기 종료



- NN이 과적합 회피하는 규제 기법

- 매 에포크의 validation loss를 측정해 모델의 종료 시점 제어
- 과적합이 발생하기 이전(validation loss , train loss )에 학습 종료
- 최고 성능 보장 ❌

✅ 콜백 함수

- 특정 이벤트 발생에 의해 자동적으로 함수를 호출하고 처리
 - 동기적 함수
 - 순차적 실행
 - 비동기적 함수
 - 병렬 처리
 - 특정 코드가 완료될때까지 기다리지 않고 너무 오래걸리면 다른 코드가 먼저 처리되도록 함